

EE 587 Introduction to Robotics

Lecturer: Prof. Afşar SARANLI

Homework 1

Student: Çağdaş Güven

Studen Number: 2738938

```
In [2]: # as of oct 31 2024 stable matplotlib version is 3.8.4 -> pip install mat
try:
    from google.colab import output
    print('Running on CoLab')
    output.enable_custom_widget_manager()
    %pip install ipympl
    %pip install spatialmath-python
    COLAB = True
    SWIFT = False
except ModuleNotFoundError:
    COLAB = False
    SWIFT = False
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "last_expr_or_assign"
from IPython.display import HTML

# standard imports
import numpy as np
from scipy import linalg
%matplotlib widget
import matplotlib.pyplot as plt
import math
from math import pi
np.set_printoptions(
    linewidth=120, formatter={
        'float': lambda x: f"{0:8.4g}" if abs(x) < 1e-10 else f"{x:8.4g}"
    })
np.random.seed(0)
from spatialmath import *
from spatialmath.base import *
```

1. Frame {A} and Frame {B} are initially coincident. Frame {B} is rotated initially around unit vector $\hat{x}^B$, by and angle θ and then around $\hat{z}^B$ by angle α . Calculate the 3×3 rotation matrix R_B^A that describes the orientation of frame {B} with respect to the frame {A}.

$$\hat{B}_{x,\theta} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c\theta & -s\theta \\ 0 & s\theta & c\theta \end{pmatrix}$$

$$\hat{B}_{z,\alpha} = \begin{pmatrix} c\alpha & -s\alpha & 0 \\ s\alpha & c\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$R_B^A = \hat{B}_{x,\theta} \hat{B}_{z,\alpha} = \begin{pmatrix} c\alpha & -s\alpha & 0 \\ c\theta s\alpha & c\theta c\alpha & -s\alpha \\ s\theta s\alpha & s\theta c\alpha & c\theta \end{pmatrix}$$

2. Suppose we have $\theta = 45^\circ$ and $\alpha = 30^\circ$. A point is given in Frame {B} with coordinate vector $p^B = [3, 4, 5]^T$. Analytically calculate its coordinate representation in Frame {A}.

$$R_B^A = \hat{B}_{x,\theta} \hat{B}_{z,\alpha} = \begin{pmatrix} c\alpha & -s\alpha & 0 \\ c\theta s\alpha & c\theta c\alpha & -s\alpha \\ s\theta c\alpha & s\theta s\alpha & c\theta \end{pmatrix}$$

$$P^A = R_B^A P^B = \begin{pmatrix} c30 & -s30 & 0 \\ c45s30 & c45c30 & -s45 \\ s45s30 & s45c30 & c45 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \\ 5 \end{pmatrix}$$

$$= \begin{pmatrix} 3\frac{\sqrt{3}}{2} + -\frac{4}{2} + 0 \\ \frac{\sqrt{2}}{2} * \frac{1}{2} * 3 + \frac{\sqrt{2}}{2} * \frac{\sqrt{3}}{2} * 4 - \frac{\sqrt{2}}{2} * 5 \\ \frac{\sqrt{2}}{2} * \frac{1}{2} * 3 + \frac{\sqrt{2}}{2} * \frac{\sqrt{3}}{2} * 4 + \frac{\sqrt{2}}{2} * 5 \end{pmatrix} = \begin{pmatrix} 0.5981 \\ -0.0254 \\ 7.0457 \end{pmatrix}$$

3. Suppose the same coordinates were given for a point q^A in Frame {A}. Calculate the coordinates of this point in frame {B}. How did you use the result in (1) to make this calculation?

So we multiply p^B with rotation matrix to find its relative position to the A frame and add it to our Homogeneous transformation 4th column.

$$H_B^A = \begin{pmatrix} R_B^A & P^A \\ 0 & 1 \end{pmatrix}$$

$$H_B^A = \begin{pmatrix} c30 & -s30 & 0 & 0.5981 \\ c45s30 & c45c30 & -s45 & -0.0254 \\ s45s30 & s45c30 & c45 & 7.0457 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

4. Now again, consider Frames {A} and {B} coincident. Frame {B} is now rotated initially around unit vector $\hat{x}^A$, by angle θ and then around $\hat{z}^A$ by angle α . Again calculate the 3×3 rotation matrix R_B^A that describes the orientation of frame {B} with respect to the frame {A}. Does your result coincide with the result in (1)? Explain.

-> Rotation w.r.t current frame result should be:

$R_2^0 = R_1^0 R_2^1 = R_1^0 (R_1^0)^{-1} R(R_1^0) = R R_1^0$ Which means Fixed Rotations are Premultiplied from the left.

$$R_B^A = B_{z,\alpha} B_{x,\theta} =$$

$$\begin{pmatrix} c\alpha & -s\alpha & 0 \\ s\alpha & c\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & c\theta & -s\theta \\ 0 & s\theta & c\theta \end{pmatrix} = \begin{pmatrix} c\theta & -s\alpha c\theta & s\alpha s\theta \\ s\alpha & c\alpha c\theta & -c\alpha s\theta \\ 0 & s\theta & c\theta \end{pmatrix}$$

$$\begin{pmatrix} c30 & -s30c45 & s30s45 \\ s30 & c30c45 & -c30s45 \\ 0 & s45 & c45 \end{pmatrix}$$

since we are multiplying our rotations in different order, the result should differ from non Fixed frame

5. Consider the basic rotations described in (1) with θ and α values given in (2).

Create the corresponding basic rotation matrices using the two alternative

toolbox functions, e.g., **SO(3).Rx()** and **rotx()** with the proper axes specified. Use

the computer to calculate the overall rotation matrix R_B^A . (Note: How do you

multiply matrices with Python3 and numpy package?) Verify your results in (2) and (3).

```
In [3]: # theta = 45      | alpha = 30
# theta = pi/4 | alpha = pi/6

theta = pi/4
alpha = pi/6
```

```
Out[3]: 0.5235987755982988
```

```
In [4]: Rx= S03.Rx(theta)
type2 = type(Rx)
print(Rx)
```

```
1      0      0
0      0.7071 -0.7071
0      0.7071 0.7071
```

```
In [5]: Rz = S03.Rz(alpha)
```

```
Out[5]: 0.866    -0.5      0
0.5      0.866      0
0         0         1
```

```
In [6]: Rotso3 = Rx * Rz
```

```
Out[6]: 0.866    -0.5      0
0.3536    0.6124  -0.7071
0.3536    0.6124  0.7071
```

```
In [7]: Rx = rotx(theta)
```

```
type1 = type(Rx)
print(Rx)
```

```
[[      1      0      0]
 [      0  0.7071 -0.7071]
 [      0  0.7071  0.7071]]
```

```
In [8]: Rz = rotx(alpha,"rad") #rad or deg
```

```
Out[8]: array([[ 0.866, -0.5,  0],
               [ 0.5,  0.866,  0],
               [ 0,  0,  1]])
```

```
In [9]: Rot = Rx @ Rz
```

```
Out[9]: array([[ 0.866, -0.5,  0],
               [ 0.3536,  0.6124, -0.7071],
               [ 0.3536,  0.6124,  0.7071]])
```

6. Identify (how in python3?) the data types of the rotation matrices obtained with the two alternative versions of creating the rotation matrices.

```
In [10]: print('Data type for rot function is:',type1, "\nfor S03 function is:",
Data type for rot function is: <class 'numpy.ndarray'>
for S03 function is: <class 'spatialmath.pose3d.S03'>
```

```
In [11]: A= rotx(0)
```

```
Out[11]: array([[ 1,  0,  0],
               [ 0,  1,  0],
               [ 0,  0,  1]])
```

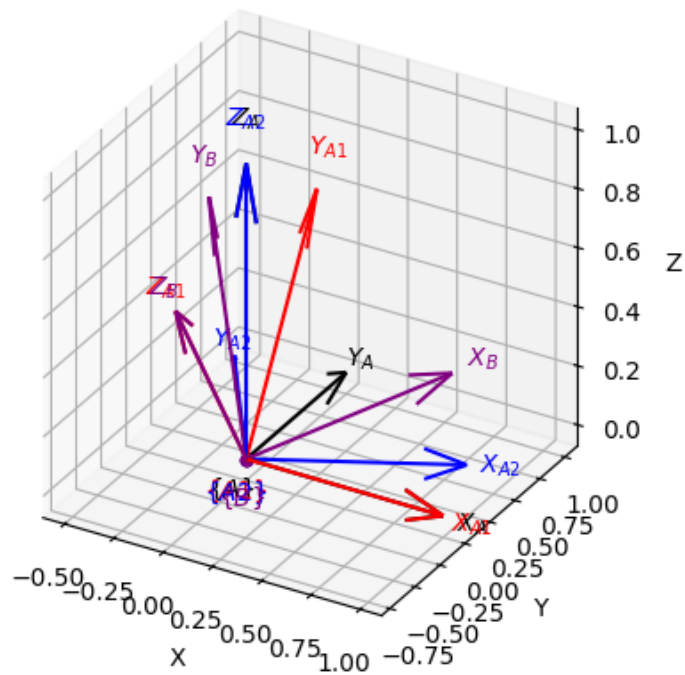
7. Explore the options of your chosen plotting function SO(3).plot() or trplot() such as specifying the Frame label, color, coordinate range etc. Now plot

- The original frame {A},
- The intermediate frames {A1 } and {A2 } resulting from the two basic rotations,
- The final Frame {B}.

```
In [12]: plotvol3(new=True)
trplot(A,color='black',frame='A') #A
trplot(Rx,color='red',frame='A1') #A1
trplot(Rz,color='blue',frame='A2') #A2
trplot(Rot,color='purple',frame='B') #B
```

```
Out[12]: <Axes3D: xlabel='X', ylabel='Y', zlabel='Z'>
```

Figure

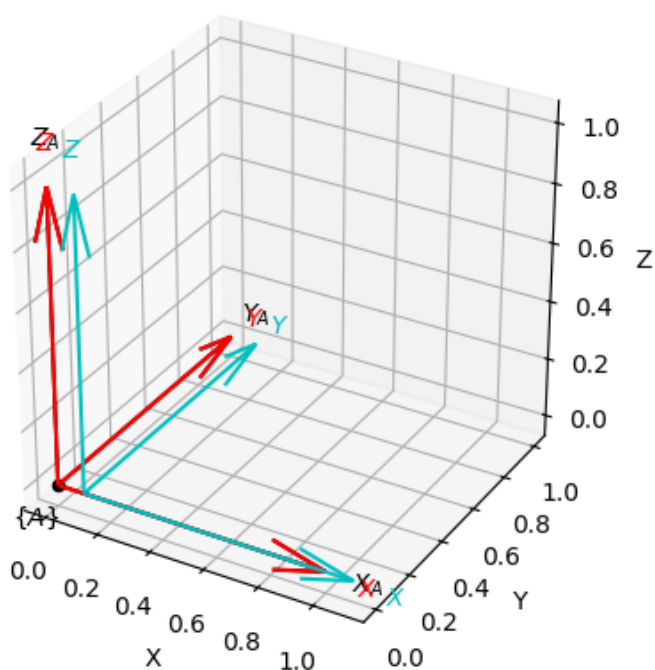


8. Use **tranimate()** to animate the overall rotation from Frame {A} to Frame {B}.
(There is also a **SO(3).animate()** version).

```
In [14]: plotvol3(new=True)
         trplot(A,color='black',frame='A')
         tranimate(Rot,anaglyph=True)
```

```
Out[14]: <matplotlib.animation.FuncAnimation at 0x7600e7716260>
```

Figure



9. Given the Homogeneous transformation matrix

$$T_A^B = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & \cos(\theta) & -\sin(\theta) & 2 \\ 0 & \sin(\theta) & \cos(\theta) & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

a. Analytically find the homogeneous transformation T_B^A .

b. Given $\theta = 45^\circ$ and a point $p^B = [3, 4, 5]^T$ in frame {B}, compute the coordinates p^A in Frame {A}.

$$H_B^A = \begin{pmatrix} R_B^A & P_B^A \\ 0 & 1 \end{pmatrix}$$

$$H^{-1} = \begin{pmatrix} R^T & -R^T \cdot P_A^B \\ 0 & 1 \end{pmatrix}$$

a.

$$\begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & c\theta & s\theta & (2c\theta + 3s\theta) \\ 0 & -s\theta & c\theta & (-2s\theta + 3c\theta) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

b.

$$P_B^A = \begin{pmatrix} 3 \\ (4c45 + 5s45) \\ (-4s45 + 5c45) \end{pmatrix} = \begin{pmatrix} 3 \\ 6.3640 \\ 0.7071 \end{pmatrix}$$

10. Show that $\in SO(3)$, namely it is a Rotation Matrix.

$$T = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \\ -\frac{1}{2} & \frac{1}{\sqrt{2}} & \frac{1}{2} \\ -\frac{1}{2} & -\frac{1}{\sqrt{2}} & \frac{1}{2} \end{pmatrix}$$

- So as a SO3 Matrix we should prove that $T^T T = I$ which means matrix is orthogonal
- Also $\det(R) = 1$ so the Matrix preveserves rotation

```
In [13]: import numpy as np

# Defining the Matrix
R = np.array([
    [1/np.sqrt(2), 0, 1/np.sqrt(2)],
    [-1/2, 1/np.sqrt(2), 1/2],
    [-1/2, -1/np.sqrt(2), 1/2]
])

# Calculate the determinant of R
det_R = np.linalg.det(R)

# Calculating T^T * T = I
identity_check = np.dot(R.T, R)

print("determinant of T Matrix is:\n", det_R, "\nidentity check gives:\n")
```

determinant of T Matrix is:

0.9999999999999998

identity check gives:

```
[[ 1.  0.  0.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]
```

which satisfies determinant = 1

Programming Task:

11. Animate a rotating cube:

a. Write a function to plot the edges of a cube centered at the origin. (Note: You will need the **mplot3d** extensions to the **matplotlib** package in Python3)

b. Modify the function to accept an argument which is a homogeneous transformation that is applied the the cube vertices before plotting.

c. Animate rotation about the x-axis.

d. Animate rotation about all axes.

```
In [ ]: # Caution ! animation eats alot CPU !
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from spatialmath import SE3
from matplotlib.animation import FuncAnimation

# Define cube vertices and edges
def cube_vertices_edges():
    # Cube vertices centered at origin
    half_size = 0.5
    vertices = np.array([
        [ half_size,  half_size,  half_size],
        [ half_size,  half_size, -half_size],
        [ half_size, -half_size,  half_size],
        [ half_size, -half_size, -half_size],
        [-half_size,  half_size,  half_size],
        [-half_size,  half_size, -half_size],
        [-half_size, -half_size,  half_size],
        [-half_size, -half_size, -half_size]
    ])

    # Define edges by pairs of vertices
    edges = [
        [0, 1], [0, 2], [0, 4], [1, 3], [1, 5],
        [2, 3], [2, 6], [3, 7], [4, 5], [4, 6],
        [5, 7], [6, 7]
    ]

    return vertices, edges

# Plot the cube with transformation applied
def plot_cube(ax, vertices, edges, transform=None):
    transformed_vertices = []

    if transform is not None:
        # Apply the transformation to each vertex
        transformed_vertices = np.array([transform @ np.append(v, 1) for v in vertices])
    else:
        transformed_vertices = vertices

    # Plot each edge
    for edge in edges:
        points = transformed_vertices[edge]
        ax.plot(points[:, 0], points[:, 1], points[:, 2], color="b")

# Animation function for rotating cube about the x-axis
def animate_rotation_x():
    fig = plt.figure()
    ax = fig.add_subplot(111, projection="3d")
    vertices, edges = cube_vertices_edges()

    def update(frame):
        ax.cla()
        ax.set_xlim([-1, 1])
        ax.set_ylim([-1, 1])
        ax.set_zlim([-1, 1])
```



```
# Create a transformation matrix for rotation about x-axis
transform = SE3.Rx(np.radians(frame))
plot_cube(ax, vertices, edges, transform.A)

ani = FuncAnimation(fig, update, frames=range(0, 360, 5), interval=50)
plt.show()

# Animation function for rotating cube about all axes
def animate_rotation_all_axes():
    fig = plt.figure()
    ax = fig.add_subplot(111, projection="3d")
    vertices, edges = cube_vertices_edges()

    def update(frame):
        ax.cla()
        ax.set_xlim([-1, 1])
        ax.set_ylim([-1, 1])
        ax.set_zlim([-1, 1])
        # Create a composite transformation matrix rotating about all axes
        transform = SE3.Rx(np.radians(frame)) * SE3.Ry(np.radians(frame))
        plot_cube(ax, vertices, edges, transform.A)

    ani = FuncAnimation(fig, update, frames=range(0, 360, 5), interval=50)
    plt.show()

# Run the animations
animate_rotation_x()
animate_rotation_all_axes()

# Caution ! animation eats alot CPU !
```